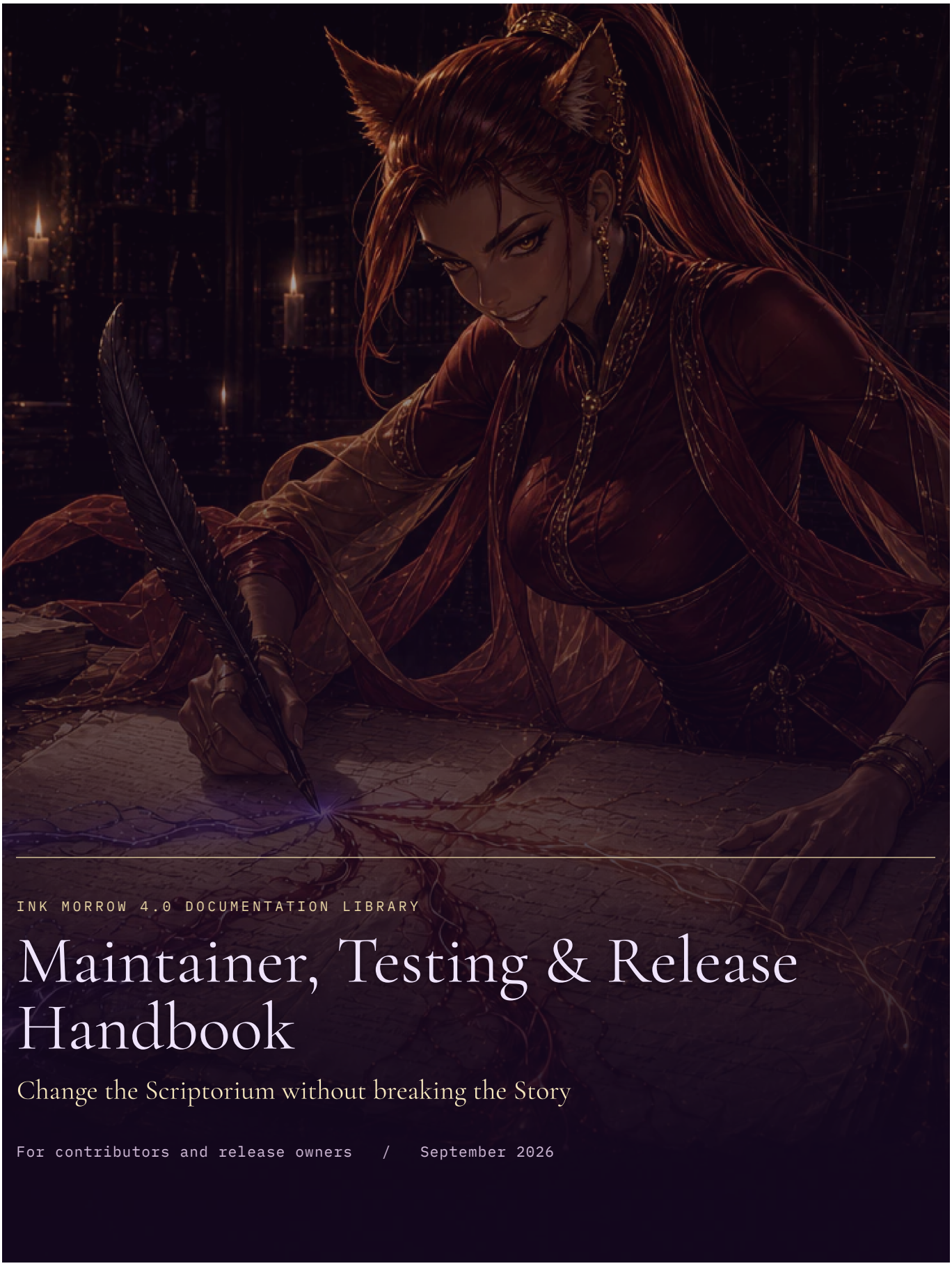

INK MORROW 4.0 DOCUMENTATION LIBRARY

Maintainer, Testing & Release Handbook

Change the Scriptorium without breaking the Story

For contributors and release owners / September 2026

Maintainer, Testing & Release Handbook

Ink Morrow grew from a playful afternoon into a serious stateful system. This handbook turns that seriousness into repeatable contribution and release practice without pretending the project has a large permanent team.

It explains how to understand a change, preserve data contracts, choose the right test layer, use automation responsibly, assemble release evidence, and stop when evidence says the change is not safe.

Guiding principle: automate the mechanical work aggressively; keep human judgment at architectural, literary, security, and release boundaries.

Contents

01 Repository orientation

Area	Responsibility
`backend/`	Express API, SQLite domain modules, providers, publication, transfer
`frontend/`	Same-origin browser application and feature modules
`e2e/`	Playwright user journeys against isolated server/database
`docs/`	Feature contracts, release evidence, user guide, PDF library
`scripts/`	Repository-level policy and validation helpers
`.github/`	CI and contribution/security surfaces
`AGENTS.md`	Persistent engineering constraints and learned project context

Read the nearest contract before changing code. For cross-cutting work, begin with system architecture, state-machine atlas, security boundary, and the feature document. Search for existing tests and migration history before inventing new behavior.

02 Change classification

Classify the proposal before implementation:

Class	Typical examples	Required attention
Presentation	Typography, spacing, copy	Responsive, accessibility, screenshots
Local behavior	UI validation, navigation	Frontend tests and stale-state handling
Domain behavior	revisions, canon, cast, jobs	Backend integration and invariants
Persistent contract	schema, archive, publication format	Migration, compatibility, rollback, fixtures
Provider boundary	models, prompts, cost, errors	Mock call counts, data review, idempotency
Trust boundary	auth, upload, share, import	Threat model and adversarial tests
Release/operations	startup, environment, CI	Fail-fast behavior and operator guidance

If a change spans classes, satisfy the highest-risk obligations. A one-line schema default is still a persistent-contract change.

03 Planning a pull request

A good PR states:

- user/operator problem and acceptance outcome;
- data/API/UI contracts that will change;
- explicit non-goals;
- state transitions and restart behavior;
- provider data and spend implications;
- migration, archive, publication, and rollback impact;
- test evidence expected at each layer; and
- documentation that must change.

Prefer one coherent vertical slice over parallel models. When replacing a flow, remove obsolete entry points and terminology in the same PR unless a compatibility boundary is deliberately documented. Dual systems become data ambiguity.

> **STATE FIRST**
>
> If work can be pending, retried, cancelled, interrupted, stale, or failed, design the durable state machine before the UI. A spinner is not a persistence contract.

04 Branch and commit discipline

Branch from the verified latest main commit. Keep unrelated user changes intact. Use narrow commits with imperative summaries. Do not rewrite published history to remove an old name or mistake; make a forward correction and preserve the audit trail.

Before push:

```
npm run check:brand
npm test
git diff --check
```

Run browser E2E locally only when the changed behavior is likely to break the primary flow, when diagnosing CI, or when creating deliberate screenshot evidence. Otherwise leave E2E to the isolated CI job as project practice.

Never point E2E at port 3000 or a real data directory. The suite owns its clean port and in-memory/isolated database.

05 Test pyramid

Pure unit and schema tests

Use for rank helpers, strict validators, deterministic folds, prompt/context selection, format escaping, redaction, token hashing, media signatures, and transition predicates. These should be fast and exhaustive around boundaries.

Backend integration tests

Use in-memory SQLite unless filesystem behavior is the subject. Cross the real transaction boundary for hierarchy/revision mutation, prepared promotion, operation idempotency, continuity, Author Canon, vault, archive preflight/commit, media staging, publication snapshots, and sharing.

Providers are mocked. Assert exact call count, model role, input categories, error classification, known usage, and absence of secret canaries.

Frontend integration tests

Jest/jsdom covers feature state, API adaptation, dialog lifecycle, accessible names, stale response rejection, and the visible consequences of backend states. Test idle/loading/success/empty/refusal/failure/cancelled/retry/stale where relevant.

Browser E2E

Playwright proves high-value composition across browser, server, and persistence:

1. first-run setup and unlock;
2. manual and AI-assisted manuscript start;
3. exact prepared-page promotion and directed replacement;
4. retry/double-click/racing-tab safety;
5. tail edit, history copyedit, truncate, undo, restore;
6. Chronicle failure/repair and Codex author corrections;
7. upload/place art without provider traffic;
8. provider refusal and deliberate retry;
9. publication formats, backup/restore, share/revoke; and
10. lock with no private flash.

06 State-machine testing pattern

For each transition, cover:

- valid predecessor and expected durable effects;
- invalid predecessor;
- repeated request/idempotency;
- concurrent owner;
- stale expected revision/context;
- provider timeout and malformed result;
- cancellation;
- restart after each durable boundary;
- visible specific error and repair route; and
- reload proving persistence.

High-risk machines deserve transition-table tests rather than only journey tests. The table should make missing edges obvious.

07 Database and migration changes

Database family and schema version are security/data-safety boundaries. A migration must be ordered, checksummed, transactional, and idempotent only in the explicitly supported sense. Startup proves identity before mutation and refuses unknown/future/3.x data.

Migration PR evidence includes:

- clean database creation;
- upgrade from every supported 4.0 predecessor fixture;
- earlier pre-rebrand adoption and backup where applicable;
- checksum mismatch refusal;
- future/old-family refusal before writes;
- restart after applied migration;
- archive/export behavior; and
- documented rollback using complete cold backup, not reverse SQL improvisation.

Do not rename a database and infer identity. Do not delete migration history. Do not weaken a refusal to make a local fixture pass.

The current 4.1.0 storage contract is schema 13: hierarchy, immutable page revisions, and revision-bound continuity deltas are canonical. The `manuscript_pages` read view is an API projection, never a writable compatibility store. Schema-12 upgrade tests must prove preservation of prose, Chronicle state, known and unknown cost semantics, and accumulated retry usage before the retired mirror tables disappear.

08 API and schema evolution

Strict schemas should reject unknown fields where accepting them would hide provider or import drift. Public/private response fields are allowlisted. New enums require state-machine documentation, restart handling, frontend rendering, tests, and export/archive consideration.

Changing an API requires synchronized backend, frontend adapter, fixtures, tests, and docs. Compatibility aliases should have a removal contract; accidental indefinite duality is not compatibility.

09 Provider work

All provider calls require an explicit role, bounded context, timeout, sanitized errors, usage recording, and data/spend review appropriate to the action. Automatic continuity uses the server-configured Archivist. Model discovery may report unavailable choices but never silently replace an explicit assignment.

Tests never spend money. Use deterministic mocks for success, delay, partial stream, refusal, invalid JSON, schema mismatch, oversized error, timeout, and late/stale response.

A paid operation is incomplete until duplicate-request and restart behavior are proven.

10 UI and accessibility

The supported browser is current Chrome, with desktop and Android tablet portrait/landscape as critical profiles. Other sane standards-respecting browsers should work but are not release evidence.

Every critical flow must remain keyboard-completable. Maintain visible focus, meaningful accessible names, status announcements, non-color-only states, dialog focus trapping/restoration, 200% text zoom, reduced motion, touch-size controls, and resilience to short viewports/on-screen keyboards.

Menus must escape clipping/stacking containers; a visible More button whose options are covered is a broken action, not cosmetic polish.

Visual review distinguishes art changes from clipping, contrast regression, obscured manuscript content, and responsive control movement.

11 Documentation as product behavior

Update the User Guide for workflows, the Operations Handbook for deployment/recovery, Architecture for boundaries/decisions, the State Atlas for new durable states, Security for trust-boundary changes, and this handbook for process changes.

PDF source and final PDFs are version controlled. Rendering must complete with fonts and images loaded, outline/bookmarks enabled, and no content clipping. Render pages to images and visually inspect the latest version before merge.

The GitHub README remains the front door, not a substitute for the guide. Link the documentation library clearly and keep setup concise.

12 Security review triggers

Update the threat model and adversarial tests when a change touches:

- login/session/CSRF/Host/origin;
- secrets or logs;
- provider endpoint/payload/result;
- upload/decode/storage;
- archive/import/export;
- public share/publication;
- database identity/migration;
- paths, filenames, or process execution; or
- third-party dependencies/Actions.

Use canaries to prove that credentials, sessions, private prose, directions, recovery material, and share tokens do not cross forbidden outputs.

13 CI pipeline

CI should independently prove:

1. lockfile install and lint;
2. old-brand residue guard;
3. backend suite;
4. frontend suite;
5. isolated Playwright desktop/tablet journeys;
6. archive/publication/security policy checks;
7. dependency/Action review appropriate to the branch; and
8. clean diff/generated artifact expectations.

A green branch means the recorded checks passed on that commit; it does not replace review of architectural fit, documentation, or real-device release smoke.

The project loop permits at most seven red CI runs for ordinary automated iteration. A red run caused by infrastructure should be classified separately but still investigated. At seven product-caused reds, stop, summarize evidence, and redesign instead of thrashing.

14 Pull request review gates

A PR is mergeable only when:

- acceptance contracts and non-goals are satisfied;
- no unrelated changes are smuggled in;
- state/restart/retry behavior is explicit;
- persistence and provider boundaries have appropriate integration tests;
- user-visible errors are specific and actionable;
- accessibility and responsive risks are addressed;
- secrets and private data remain excluded;
- docs and format contracts are current;
- local required tests pass; and
- all GitHub checks are green on the final commit.

Merge through GitHub after green checks. Preserve the merge/PR audit trail. Do not bypass protection merely because local tests passed.

15 Release candidate preparation

Freeze the candidate commit and record:

- commit/tag, Node version, Chrome version;
- desktop/tablet device profiles;
- database family/schema/migration ledger;
- tested provider and model roles;
- dependency audit disposition;
- automated check URLs/results;
- manual smoke evidence;
- documentation/PDF versions;
- known issues and workarounds; and
- explicit release decision.

No code or documentation changes occur after evidence capture without creating a new candidate and rerunning affected gates.

16 Manual release smoke

On a clean self-hosted installation:

1. follow README without undocumented steps;
2. setup, lock/unlock, restart, change password;
3. create manuscripts manually, from foundations, and import;
4. edit title and Author Canon facts/world events;
5. exercise generate/prepare/direct/cancel/fail/retry/race;
6. edit tail, copyedit history, truncate, undo, recover;
7. inspect Chronicle and repair one explicit memory failure;
8. correct state and review impact in Codex;
9. upload/place personal art without provider traffic;
10. exercise image refusal and deliberate retry;
11. narrate and build representative audiobook if supported;
12. full backup, isolated restore, and semantic comparison;
13. export every publication format and open representative files;
14. publish, signed-out read, expire, and revoke a snapshot;
15. run long-manuscript smoke on reference tablet; and
16. inspect logs/artifacts for secret/private-data canaries.

17 Long-manuscript qualification

The deterministic release fixture represents 10 volumes, 100 chapters, at least 3,000 pages, about 1.2 million words, 150 recurring characters, 10,000 continuity records, 500 media records, copyedits, corrections, prepared work, and recovery state.

Prove that Desk open/page turn/save, continuity fold, bounded context construction, and Chronicle open do not scale with full manuscript text. Round-trip the project archive and publication semantics. Verify no AI operation requires the whole novel.

Record reference tablet performance. Regressions above 20 percent require explanation and stakeholder acceptance; correctness and data safety never yield to a speed target.

18 Release blockers

Do not release a reproducible defect that can cause:

- manuscript/hierarchy/revision/continuity/recovery/archive/media loss or silent corruption;
- speculative or unseen prose becoming canon;
- duplicate/unattributed known provider spend;
- stale results mutating another context;
- credential/session/private prose/recovery/share-token disclosure;
- upload-triggered AI without consent;
- archive/upload parser escape or active execution;
- public snapshot mutation/private API access;
- same-version restore failure or invalid published formats;
- inability to complete the primary flow on desktop/tablet Chrome;
- authentication bypass or known critical/high vulnerability; or
- inaccessible critical action without an equivalent path.

Lower-severity issues require a documented workaround and explicit acceptance.

19 Incident and regression handling

Preserve the failing commit, exact test/log, environment, fixture identity, and sanitized evidence. Reproduce at the lowest useful layer. Distinguish product defect, flaky test, provider drift, and CI infrastructure.

Never weaken an assertion merely to fit new behavior. First state the old invariant, the intended new invariant, and why the change is safe. Add the regression test before or with the fix.

For data-risk incidents, stop writes, make a cold copy, and investigate on a duplicate. For secret exposure, revoke/rotate first and then clean outputs; Git history remains an audit trail unless a separate high-risk remediation is explicitly approved.

20 Automation and human control

Codex can inspect, implement, test, generate assets and documentation, push branches, create PRs, watch CI, and iterate fixes. Automation reduces the cost of maintaining a one-person project, but it does not broaden authorization.

Human approval remains essential for feature direction, art/voice, destructive or public operations, secrets, nonstandard main/release integration, and final release acceptance. The automation should expose uncertainty and evidence, not manufacture confidence.

Creator's note

> *Somewhere along the way this went from a fun afternoon with a tablet to a rather serious project. I've developed things professionally since 1995, so maybe the stripes are permanent now. Anyway, I have a day job. But Codex does not. So. As full automation as possible it is. Though I remain involved. To a degree that only the wife actually creates any stories. Oh well.*

21 Definition of done

5.0 delivery train

The owner approved a new playable-fiction product on `release/5.0.0`, with no requirement to open older databases or saved stories. Substantial feature PRs target that branch and merge only after their current head passes CI. This does not authorize deployment over the running 4.x instance or integration into main. The implementation plan and actual batch evidence live in `docs/releases/5.0.0/README.md`. Reader-director play is the default; an avatar is never required. Character inhabiting must remain an explicit optional handoff.

The foundation suite tests branch-local promises/resources/control, hidden-state filtering, immutable evidence, paging, ending/resuming episodes, paid idempotency, overlapping work, rollback on invalid effects, restart interruption, and real authentication/CSRF routing. Later UI/director/portability batches must add their own checks rather than treating these backend tests as end-to-end proof.

The reader batch adds twelve component tests for no-avatar creation, busy gating, cancel/failure drafts, route/lock races, explicit control, plaintext rendering, episode rest and real provider capabilities. Fourteen active browser tests cover the complete reader journeys, WCAG A/AA checks and narrow-screen reflow, on desktop and mobile Chrome. Superseded 4.x browser contracts are archived in the release folder; backend and component regressions remain active. Use installed Chrome or the CI browser image: do not download an additional browser for local validation.

A change is done when code, schema, migrations, tests, documentation, generated artifacts, and operator expectations describe the same system; local required checks and final CI are green; review evidence is attached; rollback is understood; and there is no safe in-scope work left unfinished.

The Scriptorium exists to serve the Story. Release machinery exists to ensure the Story survives the machinery.